

Nesneye Y6nelik Programlama

UYGULAMA 10-11

Do. Dr. Emir 6ZT6RK

Arş. G6r. Oğuz KIRAT

Arayüz (Interface) Nedir?

- Sınıfların uyması gereken bir SÖZLEŞME'dir.
- "Bunu yapabiliyor musun?" sorusuna cevap verir.
- interface anahtar kelimesi ile tanımlanır.
- Bir sınıf BİRDEN FAZLA arayüzü uygulayabilir (çoklu kalıtım).
- Java 8+ ile default ve static metotlara izin verir.
- Tüm metotlar varsayılan olarak public abstract'tır.
- İlişki: "can-do" (... yapabilir).

Arayüz — Sözdizimi

```
public interface Yazdirilabilir {
    // Soyut metot – public abstract gizli
    void yazdir();

    // Default metot (Java 8+) – gövdesi VAR, isteğe bağlı override
    default void yazdirVeImzala() {
        yazdir();
        System.out.println("--- imza ---");
    }

    // Static metot da olabilir
    static String varsayilanFormat() {
        return "TXT";
    }
}

// Kullanımı:
public class Rapor implements Yazdirilabilir {
    @Override
    public void yazdir() { System.out.println("Rapor yazdırıldı."); }
}
```

Arayüz — Önemli Kurallar

- Bir arayüzden new ile nesne oluşturulamaz.
- Arayüzde tanımlı tüm field'lar otomatik public static final.
- implements ile uygulanır; class A implements B, C { ... } ← ÇOKLU KALITIM!
- Arayüzler birbirinden extends ile türeyebilir (genişler).
- Bir sınıf bir arayüzdeki TÜM soyut metotları yazmak zorundadır.
- Default metotlar arasında çakışma olursa (çoklu kalıtım) override ile çözülmelidir.

Soyut Sınıf (Abstract Class) Nedir?

- Doğrudan nesne (instance) oluşturulamayan sınıftır.
- "Yarı tamamlanmış bir şablon" gibi düşünün.
- abstract anahtar kelimesi ile tanımlanır.
- Hem normal (concrete) hem de soyut (abstract) metot içerebilir.
- Soyut metotların gövdesi yoktur — alt sınıflar yazmak zorundadır.
- Alt sınıflar arasındaki ortak DURUM ve davranışı paylaşmak için kullanılır.
- İlişki: "is-a" (bir tür ... dır).

Soyut Sınıf — Sözdizimi

```
public abstract class Calisan {  
    protected String ad;          // Ortak alan  
    protected double maas;  
  
    public Calisan(String ad, double maas) {  
        this.ad = ad;  
        this.maas = maas;  
    }  
  
    // Concrete (normal) metot – herkes aynı şekilde çalışır  
    public void bilgileriYazdir() {  
        System.out.println(ad + " - " + maas + " TL");  
    }  
  
    // Abstract (soyut) metot – gövdesi yok!  
    // Alt sınıflar mutlaka kendisi yazmak zorunda.  
    public abstract double primHesapla();  
}
```

Interface Field

Implement edilmez. Otomatik olarak public static final → sabittir.

```
public interface Vehicle {  
    int MAX_SPEED = 200;    // implicit: public static final  
}  
  
public class Car implements Vehicle {  
    void test() {  
        System.out.println(MAX_SPEED);    // 200  
        // MAX_SPEED = 250;    // ❌ HATA: final, atanamaz  
    }  
}
```



Tüm interface field'ları SABİTTİR. Değiştiremez, override edemezsin — sadece okur ve kullanırsın.

Interface Abstract Method

Concrete (somut) class bu metodu IMPLEMENT etmek ZORUNDADIR.

```
public interface Vehicle {
    void start();    // abstract – gövdesi yok
}

public class Car implements Vehicle {
    @Override
    public void start() {                // ✅ zorunlu implement
        System.out.println("Car started");
    }
}
// implement edilmezse → derleme hatası
```



Implement edilmezse program derlenmez. Bu, interface'in en katı kuralıdır..

Implement etmek zorunda DEĞİL. İsterse class override eder. (Java 8+)

```
public interface Vehicle {
    default void honk() {
        System.out.println("Beep!");
    }
}

public class Car implements Vehicle {
    // honk() override edilmedi → default kullanılır
}

public class Truck implements Vehicle {
    @Override
    public void honk() { System.out.println("HOOONK!"); } // ✅ override
}
```



Hazır gövde miras alınır. Override OPSİYONELDİR — yeterli değilse değiştirebilirsin.

Class'a miras kalmaz. Interface adıyla çağrılır. (Java 8+)

```
public interface Vehicle {  
    static void info() {  
        System.out.println("Vehicle interface");  
    }  
}  
  
public class Car implements Vehicle { }  
  
// Kullanım:  
Vehicle.info();    //  interface adıyla  
// Car.info();     //  HATA: Car'a miras kalmaz
```



Static metotlar interface'e bağlıdır, class'lara değil. Override de edilemez.

Interface Private Method

Sadece interface içinden çağrılır. Default metotlar için yardımcıdır. (Java 9+)

```
public interface Vehicle {  
    private void log(String msg) {           // private helper  
        System.out.println("[LOG] " + msg);  
    }  
  
    default void startWithLog() {  
        log("starting...");                 // ✅ içeriden çağrı  
        System.out.println("started");  
    }  
}  
  
public class Car implements Vehicle { }  
// Car içinden log(...) → ❌ erişilemez
```



Sadece kod tekrarını önlemek için. Class implement etmez, dışarıdan görmez.

Abstract Class (Soyut Sınıf)

Alanlar Subclass tarafından doğrudan MİRAS alınır.

```
public abstract class Animal {  
    protected String name;           // miras alınır  
    public static int count = 0;      // miras alınır  
  
    public Animal(String name) { this.name = name; }  
}  
  
public class Dog extends Animal {  
    public Dog(String name) { super(name); }  
    public void greet() {  
        System.out.println("I am " + name); // ✅ miras  
    }  
}
```



Field'lar subclass'a doğrudan geçer. Static, final, instance — her tür field tutulabilir.

Abstract Class Concrete Method

Implement etmek zorunda DEĞİL. İsterse subclass override eder.

```
public abstract class Animal {  
    public void sleep() {                // concrete – gövdesi var  
        System.out.println("Sleeping...");  
    }  
}  
  
public class Dog extends Animal {  
    // sleep() override edilmedi → miras alınan versiyon kullanılır  
}  
  
public class Cat extends Animal {  
    @Override  
    public void sleep() {                // ✅ opsiyonel override  
        System.out.println("Cat is sleeping");  
    }  
}
```



Interface'in default metoduna çok benzer: hazır davranış miras alınır, gerekirse değiştirilir.

Concrete subclass IMPLEMENT etmek ZORUNDADIR.

```
public abstract class Animal {  
    public abstract void sound();    // abstract – gövdesi yok  
}  
  
public class Dog extends Animal {  
    @Override  
    public void sound() {            // ✅ zorunlu implement  
        System.out.println("Woof!");  
    }  
}  
// Dog implement etmezse → kendisi de abstract olmak zorunda
```



Interface abstract metoduyla aynı kural: somut subclass implement etmek zorunda.

Hangisini Ne Zaman Kullanmalıyız?

ABSTRACT CLASS:

Birkaç sınıf benzer ve ortak alan/davranış paylaşıyorsa.

Örnek: Otomobil, Kamyon, Motosiklet → ortak "Tasit" sınıfı.

INTERFACE:

Birbirinden çok farklı sınıflar AYNI yeteneğe sahipse.

Örnek: Telefon, Otomobil, Kamera → hepsi "Sarjedilebilir".

Interface ile başlayıp, abstract sınıfa dönmek genellikle daha kolaydır.

Soyut Sınıf vs Arayüz

Özellik	Abstract Class	Interface
Anahtar kelime	abstract class	interface
Kalıtım yönü	extends (TEK sınıftan)	implements (ÇOKLU arayüz kalıtımı var)
Constructor	Var	Yok
Field (alan)	Her tür mümkün	Sadece public static final
Metot türleri	abstract + concrete	abstract + default + static
İlişki türü	is-a (bir tür ... dır)	can-do (... yapabilir)
Ne zaman?	Ortak DURUM paylaşımı	Yetenek/sözleşme

Üye Türü	Container	Implement Zorunlu?	Notlar
Field	Interface	— Hayır	Otomatik public static final (sabit)
Abstract method	Interface	✓ ZORUNLU	Concrete class implement zorunda
Default method	Interface	○ Opsiyonel	Hazır gövde; istersen override (Java 8+)
Static method	Interface	— Hayır	Interface adıyla çağrılır (Java 8+)
Private method	Interface	— Hayır	Sadece interface içinde yardımcı (Java 9+)
Field	Abstract Class	— Hayır	Subclass miras alır
Concrete method	Abstract Class	○ Opsiyonel	Hazır gövde; istersen override
Abstract method	Abstract Class	✓ ZORUNLU	Concrete subclass implement zorunda

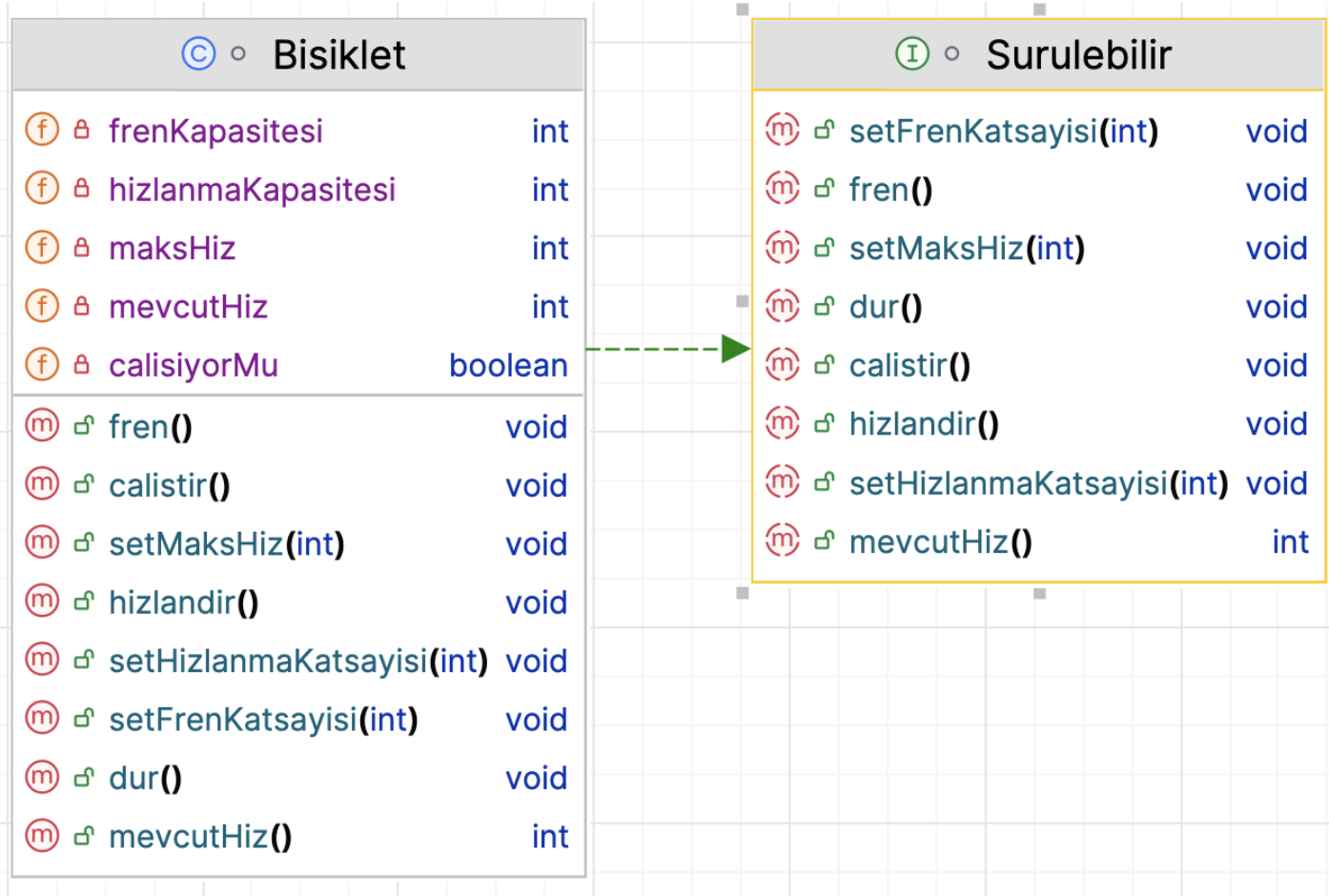
ZORUNLU → derleme hatası verir. Opsiyonel → hazır gövde vardır, istersen override edersin.

Surulebilir

 	setFrenKatsayisi(int)	void
 	fren()	void
 	setMaksHiz(int)	void
 	dur()	void
 	calistir()	void
 	hizlandir()	void
 	setHizlanmaKatsayisi(int)	void
 	mevcutHiz()	int

Yandaki arayüz (interface)'i hazırlayınız.

```
interface Surulebilir {  
    void setMaksHiz(int maksHiz);  
    void setFrenKatsayisi(int frenKapasitesi);  
    void setHizlanmaKatsayisi(int hizlanmaKapasitesi);  
    void calistir();  
    void hizlandir();  
    void fren();  
    void dur();  
    int mevcutHiz();  
}
```



```
public class Main {
    public static void main(String[] args) {
        Bisiklet bisiklet = new Bisiklet();
        bisiklet.setMaksHiz(30);
        bisiklet.setFrenKatsayisi(5);
        bisiklet.setHizlanmaKatsayisi(10);

        bisiklet.calistir();
        bisiklet.hizlandir();
        bisiklet.fren();
        bisiklet.hizlandir();
        bisiklet.dur();

        System.out.println("Mevcut hız: " +
            bisiklet.mevcutHiz());
    }
}
```

Oluşturduğunuz interface'i kullanan Bisiklet sınıfını yazınız. Main metodu sağda verilmiştir.

Arayüz 2 - Ödev

- `ciz()` metodunun implementasyonu zorunlu olacak `Cizilebilir` interface'ı tanımlayın ve buradan `Kare` ve `Dikdörtgen` sınıfı tanımlayın.
- Bir `Cizilebilir` dizisi oluşturun. Bu diziye birden fazla şekil atayın.
- Dizideki tüm elemanları ekrana çizebilen bir kod yazın.